Session 05 — Project Guide

TaskFlow: React + TypeScript Foundations

This guide documents the official **Session 05 End** state of TaskFlow. It does not cover homework answers.

What's new since Session 04C

Session 04C ended with a complete TaskFlow application built in plain TypeScript — HTML, CSS, and hand-written functions that found and updated elements on the page directly. Session 05 rebuilds TaskFlow's core visual shape in **React + TypeScript**. React does not replace TypeScript — the application logic and components are still TypeScript/TSX, returning page markup directly instead of finding and updating existing markup by hand. The project also contains familiar HTML/CSS (`index.html` , `src/index.css`) and some Vite/TypeScript configuration files.

From `index.html` to the screen

```
index.html   →   main.tsx   →   App.tsx   →   components   →   UI
```

`index.html` now contains one empty element, `<div id="root"></div>` . `main.tsx` finds that element and connects React to it:

```
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

const rootElement = document.getElementById("root");

if (rootElement) {
    createRoot(rootElement).render(<App />);
}
```

The null check here is the same discipline used throughout the TypeScript phase — check before you use, no shortcuts, no non-null assertion (`!`).

What a component is

A component is a normal function that returns TSX. Not every component needs props — `Header` doesn't, because it never shows different data:

```
function Header() {
    return (
        <header className="header">
            <h1>TaskFlow</h1>
            <p>All your tasks in one place.</p>
        </header>
    );
}

export default Header;
```

`StatCard`, by contrast, genuinely needs data from whoever uses it, because it's shown three times with three different numbers:

```
interface StatCardProps {
    label: string;
    value: number;
}

function StatCard(props: StatCardProps) {
    return (
        <div className="stat-card">
            <p className="stat-label">{props.label}</p>
            <p className="stat-value">{props.value}</p>
        </div>
    );
}
```

`StatCardProps` describes exactly what this component needs from whoever uses it — the same interface pattern already used for `Task` and `User`, now describing a component's inputs instead of API data. `props.label` / `props.value` are read directly, with no destructuring.

A parent gives a child typed data like this:

```
<StatCard label="Total Tasks" value={3} />
```

Components built this session

- `Header` — no props, fully static.
- `StatCard` — typed `label: string` / `value: number` props, used three times with different data:

```
<StatCard label="Total Tasks" value={3} />
<StatCard label="Completed" value={1} />
<StatCard label="Pending" value={2} />
```

- `TaskItem` (built in teamwork) — typed `title` / `ownerName` / `statusText` / `statusClass` props, used three times to render three different task rows.

Same component + different props = reusable UI. This is the idea behind every component that genuinely needs props today.

Static, inert controls

The filter buttons and search box are back on the page, visually identical to Session 04C's, but they do nothing yet — no `onClick`, no `onChange`. They live as plain JSX directly inside `App`, not their own components, because making them interactive is next session's lesson.

What is NOT here yet

No `useState`. No event handlers. No `.map()`. No arrow functions. No callback props. No data fetching. No Fragment — every component that needs to group more than one element uses a plain `<div>`. Every value on this page is typed directly into the JSX — nothing is calculated or remembered.

The important idea to carry forward is that a component is a function, and a parent gives a child typed data through props. Next session, these same pieces of UI start to remember and change.